PROJECT REPORT

Apex Care Technologies: from a static brochure site to a database-driven web application

Design, implementation and evaluation of a full-stack rebuild of apex-caretechnologies.com using Next.js 16 and MongoDB.

Submitted by	**Manmay Chakraborty**
Registration no.	**24BAI1316**
Date	**20 October 2026**

72
API operations
27 public · 45 admin

17
MongoDB collections
plus GridFS storage

300
automated tests
28 in a real browser

0
known production
dependency advisories

Contents

1. Abstract

2. Introduction and motivation

3. Objectives

4. Analysis of the existing website

5. Proposed system and architecture

6. Technology choices and justification

7. Database design

8. Features implemented

9. Security and privacy

10. Comparison: existing vs. proposed

11. Testing and validation

12. Limitations and future work

13. Conclusion

A. Appendix: API summary and how to run

1. Abstract

Apex Care Technologies is a supplier of fire-safety, personal-protective, welding and gas-detection equipment based in Cuddalore, Tamil Nadu. Its existing website is a set of hand-written static HTML pages: all content is typed directly into the page files, an enquiry is sent to a single server script, and there is no way for the business to edit content, track customer enquiries or measure visitors without a developer.

This project rebuilds the website as a full-stack application. A single **Next.js 16** application serves both the public pages and a documented **REST API**, and **MongoDB** is the only data store: product catalogue, enquiries, users, sessions, analytics and uploaded files. The result provides 72 API operations over 17 collections, a layered anti-spam and security design, role-based administration, enquiry tracking for customers, a double opt-in newsletter, moderated testimonials, first-party privacy-preserving analytics, full-text search, search-engine optimisation, and a content backup and restore facility.

The system is verified by **300 automated tests**, of which 28 drive a real Chrome browser through the user interface. This report explains why the rebuild was needed, how it was designed, what was built, and how it improves on the original site. It also states honestly what is not yet done, chiefly that administration is currently available through the API only and does not yet have a graphical interface.

2. Introduction and motivation

For a small industrial supplier, the website is the main route by which new customers find the company and ask for a quotation. Buyers of safety equipment typically look for a specific item (for example a self-contained breathing apparatus or a gas detector), compare options and then ask for a price. The site therefore has three jobs: present the catalogue accurately, make it easy to ask for a quote, and let the business respond quickly and keep a record of every lead.

2.1 Why a rebuild was needed

A brochure site built from static files can show information, but it cannot manage it. The practical consequences for the business are:

- **Every content change needs a developer.** Adding a product, correcting a phone number or publishing a new training programme means editing HTML files and re-uploading them.
- **Leads are fragile.** An enquiry depends on a single script and, as far as a visitor can see, is not given a reference number, cannot be followed up by the customer and has no status.
- **No visibility.** The business cannot see how many people visit, which products they view, what they search for, or what fraction of visitors send an enquiry.
- **Weak discoverability.** Individual products have no pages of their own, so they cannot be found, linked or shared, and search engines are given little structured information.
- **Open to abuse.** Public forms attract automated spam; without defences, the sales inbox fills with junk and genuine leads are lost.

2.2 Scope of this project

The visual design and page set of the original site were kept recognisable so that the business is not asked to relearn its own website. The engineering effort went into what lies underneath: a real data layer, a secure API, the business workflows around enquiries, and the quality assurance needed to trust them.

3. Objectives

1. Store **all** site content in a database and render every page from it, so that an edit is visible on the live site immediately.
2. Give each product, survey and training programme its own addressable page with proper search-engine metadata.
3. Capture every enquiry reliably, number it, notify the sales team by email, acknowledge the customer, and let the customer check progress.
4. Protect the public forms against spam and abuse without inconveniencing real visitors.
5. Provide secure, role-based administration for content, enquiries, users and settings.
6. Measure site usage without tracking individuals, and respect visitor privacy signals.
7. Make the whole system easy to run, easy to back up and demonstrably correct through automated testing.

4. Analysis of the existing website

The live site at apexcaretechnologies.com was inspected on 20 October 2026, using its public pages, response headers and the scripts it serves. Only what is observable from outside is reported here; the server-side behaviour of its enquiry script, in particular, cannot be inspected and is not assumed.

Aspect	Observation on the existing site
Construction	Static HTML files (`index.html` , `products.html` , `survey.html` …) with Bootstrap and jQuery; content is written into the page files.
Product pages	All product categories link to one general `products.html` ; there is no individual page, address or search-engine entry per product.
Enquiry submission	A browser script validates the form and posts it to `enquiry.php` . Phone validation accepts only exactly ten digits (`/^\d{10}$/`), so `+91` or international numbers are rejected. The form has no email field, so the customer can never be answered or acknowledged by email.
Enquiry follow-up	The confirmation says only “We’ll be in touch”. No reference number is shown; there is no status or tracking page.
Spam control	No honeypot, token or rate limit is visible in the client code.
Search	None.
Search-engine files	`/sitemap.xml` and `/robots.txt` both return 404; no structured data was found on the home page.
Contact details	Email addresses are hidden by Cloudflare’s obfuscation script; they are not part of any editable settings.
Content administration	No administration area or content-management system was found.
Analytics, newsletter, testimonials	None visible.
Copy quality	Inconsistent capitalisation in menus (“survey”, “contact Us”) and typos in the form labels.

Table 1. Observed characteristics of the existing website.

> **Conclusion of the analysis.** The existing site is adequate as an online brochure but has no content management, no lead management, no visibility into usage and limited discoverability. These are limits of its architecture, not of its design, and they are what the rebuild addresses.

5. Proposed system and architecture

The proposed system is a single deployable application. Next.js renders the public pages on the server and exposes the REST API from the same codebase; both talk to one MongoDB database. There is no separate back-end service, no third-party content-management system and no second data store to keep in step.

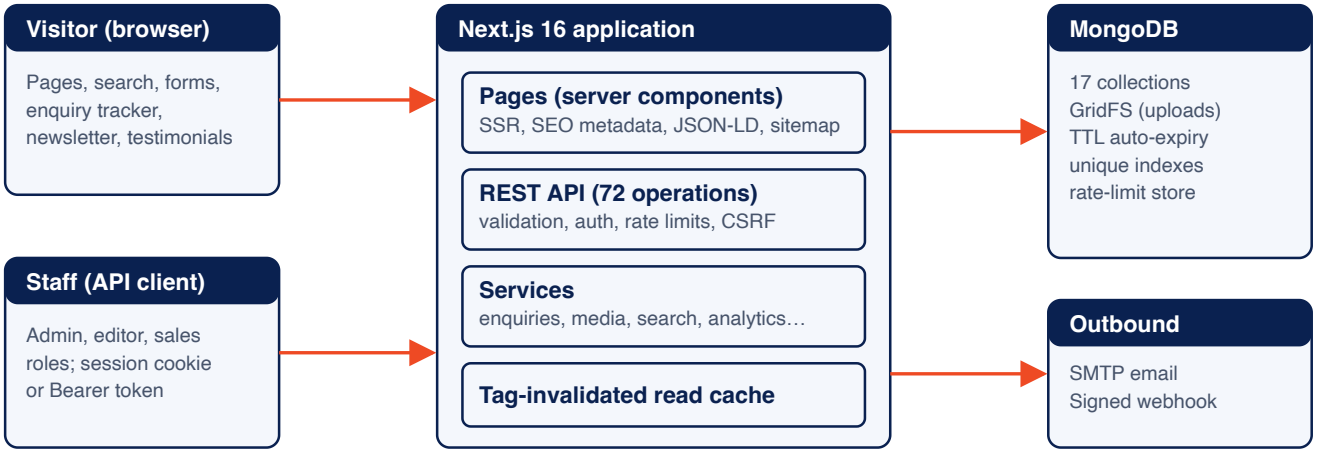


Figure 1. High-level architecture. Visitors and staff use the same application; MongoDB is the single source of truth.

5.1 Layering

Layer	Responsibility
Pages (`src/app`)	Server-rendered pages: Home, About, Products, Product detail, Survey, Survey detail, Training, Contact, 404 and error pages; `sitemap.xml` and `robots.txt` generated from the database.
API routes (`src/app/api`)	56 route files exposing 72 operations with a uniform response format and request ids.
Validation (`src/server/validation`)	Every request body and query string is checked against a strict schema; unknown fields are rejected.
Services (`src/server/services`)	Business logic: enquiries, notifications, media, search, analytics, audit, import and backup, users, settings, dashboard.
Data (`src/server/db` , `data`)	Mongoose models with indexes, and cached read functions that are invalidated by tag when content changes.
Cross-cutting (`src/server/lib` , `auth` , `mail`)	Request wrapper, errors, rate limiting, signed tokens, password hashing, sessions, permissions, mail templates.

Table 2. Application layers and their responsibilities.

5.2 How a request is handled

Take a customer enquiry as an example. The page fetches a short-lived signed token; the visitor fills in the form; the browser posts it with the token. The server checks the token and honeypot, validates and normalises the fields, scores the content for spam, applies per-address limits, allocates a reference number atomically, stores the enquiry and returns the reference. Email to the sales team, the acknowledgement to the customer and the optional webhook are then sent, and each channel records its own status, attempt count and last error so that a failure is visible and can be retried.

6. Technology choices and justification

Technology	Role	Why it was chosen
Next.js 16 (App Router)	Website and API in one application	Server rendering gives fast first loads and good search-engine indexing; route handlers remove the need for a separate API server; one deployment.
React 19	User interface	Component model suits the repeated cards, forms and dialogs; server components keep database access off the client.
MongoDB 8 + Mongoose 9	Single data store	Catalogue items have varied attributes (specifications, tags, galleries) that fit documents naturally; TTL indexes expire data automatically; GridFS stores uploads in the same database, so there is one thing to back up.
Zod 4	Validation	One schema defines both the check and the TypeScript type, so the API contract cannot drift from the code.
TypeScript 5	Language	Type checking across API, services and UI catches mismatches at build time.
Tailwind CSS 4	Styling	Consistent design tokens, responsive layout and dark-mode support.
Nodemailer	Email	Standard SMTP client; a local mail catcher is used in development so emails can be inspected without sending any.
Node crypto (scrypt, HMAC, SHA-256)	Security primitives	Built-in, audited primitives instead of extra dependencies.
Node test runner + Playwright core	Testing	Unit and API tests run against a real throw-away database; browser tests drive the installed Chrome.

Table 3. Technology stack.

6.1 Why one database for everything

An earlier plan used a hosted content-management service alongside a database. It was dropped: two stores mean two sets of credentials, two backup procedures and the risk of content and enquiries disagreeing. Keeping content, enquiries, users, analytics and files in MongoDB gives one consistent system, one backup and one place to look when something goes wrong.

6.2 Why not a CMS or website builder

A ready-made CMS would give an editing screen quickly but would constrain the enquiry workflow, tracking, spam handling and privacy-preserving analytics that the business actually needs, and would bring its own maintenance and security burden. Building on a framework keeps those behaviours exactly as specified and testable.

7. Database design

The database has 17 collections and a GridFS bucket for files. Slugs and reference numbers are unique-indexed; operational collections carry TTL indexes so that old data removes itself.

Collection	Holds	Notable design points
products	Catalogue items	Unique slug; belongs to a product group; featured flag, ordering, publish state, description, features, specification table, tags, gallery, SEO overrides
productgroups	Categories (5)	Cannot be deleted or re-keyed while products reference them
surveys	Survey/instrument pages (9)	Own pages, ordering, publish state
trainings	Training programmes (10)	Own page content and ordering
galleries · logos · leaders	Gallery, client and brand logos, leadership	Ordered, publishable
testimonials	Customer feedback	Public submissions stored unpublished; submitter email kept private
sitesettings	Singleton settings	Company details, phones, emails, address, map, social links, tax numbers, About text, home-page copy, catalogue file
enquiries	Leads	Reference number, status history, priority, assignee, tags, internal notes, per-channel notification state; spam rows auto-deleted after 30 days
subscribers	Newsletter	Pending/confirmed/unsubscribed; unconfirmed rows expire after 14 days
adminusers · sessions	Staff and logins	Only a hash of each session token is stored; sessions expire automatically
analytics	Usage events	No raw IP addresses; daily-rotating visitor hash; expires after 400 days
audit	Change history	Who changed which fields (not the values); expires after 730 days
counters · ratelimits	Infrastructure	Atomic yearly enquiry counter; request windows stored in the database so limits hold across servers and restarts
media.files / media.chunks (GridFS)	Uploaded images and PDFs	De-duplicated by SHA-256; usage-tracked

Table 4. Collections.

7.1 Initial data

An idempotent seeding script loads the original site's content into the database (39 products, 5 groups, 9 surveys, 10 training programmes, 8 gallery items, 19 logos, 3 leaders and the site settings) and creates the first administrator. Running it again restores anything deleted but never overwrites edits.

8. Features implemented

8.1 Database-driven public website

- Every page reads live data; no content is hard-coded. Home-page statistics (counts, years in business) are computed from the database.
- Reads are cached and invalidated by tag, so an administrator's edit appears on the site, in the sitemap, in search and in the enquiry drop-downs without redeploying.
- Each product, survey and training item has its own page and address, with its own title, description, canonical link and OpenGraph data.
- Structured data (`Organization`, `Product`, `Service` JSON-LD), a database-generated `sitemap.xml` and a `robots.txt`.
- Friendly 404 and error pages; responsive layout, dark mode, an image lightbox and a mobile action bar for call, WhatsApp and products.

8.2 Enquiry management (the core business feature)

- **Form on every relevant page** (Home, Contact, Product, Survey, Training). The subject is checked against real published items.
- **Validation:** Indian mobile numbers accepted with or without `+91` / `0` and normalised to `+91…`; international numbers accepted; email optional but verified when given.
- **Reference numbers** such as `ENQ-2026-000123`, allocated atomically per year so two simultaneous enquiries can never share one.
- **Notifications by channel:** email to sales (with the customer as Reply-To), an acknowledgement to the customer containing the reference, and an optional HMAC-signed webhook. Each records status, attempts and last error; failed deliveries are retried (maximum five attempts) by an administrator action or a scheduled job.
- **Customer tracking:** reference plus mobile number shows a friendly status (Received, In progress, Quote sent, Closed) and timeline. Internal notes and assignee are never exposed, and a wrong reference and a wrong mobile give identical answers so that references cannot be probed.
- **Staff triage (API):** filtering by status, type, priority, assignee, date and free text; sorting and pagination; status workflow with full history and first-response time; priority, assignment, tags and internal notes; bulk update; admin-only deletion; CSV export protected against spreadsheet formula injection.

8.3 Anti-spam without annoying real customers

Defences are layered and silent. A signed form token proves the page was loaded and enforces a minimum fill time; a hidden honeypot field catches simple bots; content scoring looks at links, spam vocabulary, disposable email domains and URLs in names; identical submissions within ten minutes collapse into one; and per-address burst and daily limits apply. A bot is told the submission succeeded, but spam is stored out of sight, never emailed, and deleted after 30 days. No CAPTCHA is shown to genuine visitors.

8.4 Customer testimonials with moderation

A "Share your experience" form on the About page stores submissions as *unpublished* and emails the team. Nothing becomes public until an administrator approves it; rejection deletes it. The submitter's email is retained for staff only.

8.5 Newsletter with double opt-in

A footer sign-up sends a confirmation email, and the address is subscribed only when its owner presses the button on the confirmation page. Links open a page rather than performing the action on a GET request, so email security scanners that “click” links cannot confirm or unsubscribe anyone. Every email carries a one-click unsubscribe. The response is identical for new, pending and already-subscribed addresses so that the list cannot be probed. Staff can list subscribers, export confirmed addresses and erase a person permanently.

8.6 Search

A live search dialog in the header searches products, surveys and training together. All terms must match, results are ranked by relevance, queries are escaped so they cannot inject patterns or cause regular-expression slowdowns, and out-of-date requests are cancelled as the visitor types.

8.7 Media library

Images (up to 8 MB) and PDFs (up to 25 MB) are stored in GridFS. The file type is determined from the file’s actual bytes rather than the name or declared type, so a renamed executable or an SVG/HTML file is refused. Identical uploads are de-duplicated, files are served with long-lived caching and ETag support, and a file still referenced by content cannot be deleted without an explicit force. The brochure PDF is downloaded through a tracked link that counts each download.

8.8 Privacy-preserving analytics

A cookie-less beacon records page views, product and survey views, and clicks on call, WhatsApp and email links; the server records catalogue downloads, searches and enquiries. No raw IP address is stored, visitors are identified only by a hash that changes daily, bots are dropped, and Do Not Track and Global Privacy Control are honoured. The administrator report gives totals, unique visitors, the enquiry conversion rate, a daily series, top pages, products, surveys and searches, call-to-action clicks, devices and referrers for any date range.

8.9 Administration

- **Generic content management for 8 content types:** list with search, filter, sort and pagination; create with automatic unique slugs; partial update; delete; reorder; bulk publish, unpublish and delete; referential integrity.
- **Dashboard:** counts, enquiries by status and type, 14-day trend, open and unassigned enquiries, failed notifications, average first-response time, recent enquiries.
- **Settings editing** (company details, contacts, About and home copy) with nested merge; map embeds restricted to Google Maps.
- **Audit log** of who changed what, and a manual cache purge.
- **Backup and restore:** a full JSON export of site content, and an import that validates the whole file first; any problem rejects the entire file, writes nothing and points to the exact offending row.

8.10 Authentication and roles

Email and password sign-in, with passwords hashed using scrypt and a policy of at least 12 characters, mixing letters and digits, and not common or derived from the email address. Sessions are opaque server-side records with a seven-day sliding expiry; a staff member can list their devices and sign out one or all others. Three roles control twelve permissions: *admin* (everything), *editor* (content, media, analytics) and *sales* (enquiries, subscriber list, analytics). The last administrator and the acting user cannot be removed or demoted.

9. Security and privacy

Threat	Measure in the system
Password theft	scrypt hashing with per-user salt; hashes never returned by the API; password change revokes other sessions.
Session theft	Random token of which only a SHA-256 is stored; HttpOnly, SameSite=Lax cookie (Secure over HTTPS); server-side revocation.
Brute force	Per-address limit, per-address-and-email limit and account lockout (10 failures, 15 minutes). Unknown emails receive the same error and take comparable time, so accounts cannot be enumerated.
Cross-site request forgery	Origin and Sec-Fetch-Site checks on cookie-authenticated writes.
Privilege escalation	Role-based permissions checked on every admin operation; role changes and deactivation end sessions.
Injection	Strict schemas; typed queries so user input cannot introduce database operators; escaped regular expressions; HTML-escaped email templates; header-injection guards; escaped JSON-LD; CSV formula neutralisation.
Malicious uploads	Content sniffing, size limits, executable/SVG/HTML refusal, `nosniff` and sandboxing CSP on served files.
Spam and flooding	Token, honeypot, scoring, duplicate collapsing and database-backed rate limits with `X-RateLimit-*` headers.
Excess data collection	No raw IPs; daily-rotating visitor hash; consent-based newsletter; erasure endpoint; TTL expiry of sessions, analytics, audit logs, spam and unconfirmed subscribers.
Outages	Database down returns a clean 503; failed emails and webhooks are recorded and retried; a health endpoint reports database and mail status.
Vulnerable dependencies	Next.js upgraded 16.3.6 to 16.4.2 to fix two advisories; `npm audit --omit=dev` reports none.

Table 5. Threats and countermeasures.

10. Comparison: existing vs. proposed

The existing column describes what is observable on the live site (section 4). The proposed column describes what is implemented and tested in this project.

Capability	Existing site	Proposed system
Content storage	In HTML files	**MongoDB; edits appear live**
Changing a product or phone number	Needs a developer to edit and upload files	**One authenticated API call; no redeploy**
Individual product pages	**No** (one shared page)	**Yes, 39 products with own address and metadata**
Site search	**None**	**Live, ranked, cross-collection**
Phone number validation	Exactly 10 digits only	**Indian formats normalised to +91; international accepted**
Customer email on enquiry	**No field**	**Optional, verified; acknowledgement emailed**
Reference number for an enquiry	**No**	**Yes, e.g. ENQ-2026-000123**
Customer can check status	**No**	**Yes, via reference + mobile**
Staff lead management	Not visible	**Status, priority, assignee, notes, history, bulk, CSV**
Spam protection on forms	None visible	**Token, honeypot, scoring, rate limits (silent)**
Failed notification handling	Unknown	**Per-channel status and retries**
Newsletter	**None**	**Double opt-in, unsubscribe, export, erasure**
Testimonials	**None**	**Public submission with moderation**
Analytics	**None visible**	**First-party, no cookies, no raw IPs, DNT honoured**
Search-engine optimisation	**No sitemap or robots.txt**; no structured data found	**Sitemap, robots, JSON-LD, canonical and OpenGraph per page**
User accounts and roles	**No admin area found**	**3 roles, 12 permissions, audit log**
Uploads and file handling	Static files	**GridFS, byte-level type check, de-duplication, usage tracking**
Backup and restore	Copy files	**Validated JSON export and all-or-nothing import**
Automated testing	None known	**300 tests incl. real-browser tests**
Graphical admin interface	**No**	**Not yet; API only (see section 12)**

Table 6. Capability comparison.

Summary. The public look and page structure remain familiar. The improvement is in everything behind them: the business can now manage its own content, capture and follow up leads reliably, understand its audience, and be found by search engines, while visitors gain search, individual product pages and a way to check their enquiry.

11. Testing and validation

Testing is automated and hermetic: `npm test` starts its own temporary MongoDB and mail catcher on free ports, seeds data, builds and serves the application, runs every suite and removes everything afterwards, so the developer's own data is never touched.

Suite	Tests	What it demonstrates
Unit	49	Slugs, phone parsing, CSV-injection guard, spam scoring, file sniffing, password hashing, tokens, permissions
Public API	38	Every public endpoint, filters, pagination, search, caching and security headers
Enquiries	38	End-to-end pipeline including real emails, spam handling, rate limits, triage, notes, bulk actions, CSV
Authentication	33	Login, enumeration resistance, lockout, sessions, CSRF, role matrix, user management
Content	35	CRUD for all content types, instant public visibility, integrity rules, settings merge
Engagement	27	Testimonials, tracking, newsletter double opt-in, import and restore
Media	16	Upload, serving, ETag, hostile uploads, usage-aware delete, PDF catalogue
Operations	20	Analytics, audit log, export, scheduled retries, access control
Infrastructure	10	TTL and unique indexes, seed idempotency, hashing at rest
Webhook	6	Signature, failure then retry then success, attempt cap
Browser (real Chrome)	28	All pages free of console errors and broken images; search; forms; tracker; newsletter; testimonial; lightbox; mobile; dark mode; SEO tags; analytics; privacy signal

Table 7. Automated test suites (300 tests in total).

11.1 Static checks

The TypeScript type-check and the linter are run against the whole project, and the production build succeeds without needing a database.

11.2 What testing has not covered

Browser tests ran in headless Google Chrome only; other browsers, assistive technologies and real-world load have not been tested. Email was verified through a local mail catcher rather than a live mail provider.

12. Limitations and future work

Limitation	Effect	Planned next step
No graphical admin interface	Staff must use the API (for example with curl or Postman) to edit content and handle enquiries.	Build an admin web application on the existing API; the API was designed for this.
Newsletter campaigns not sent	Sign-ups, consent and export exist; sending does not.	Export confirmed addresses to a mailing tool, or add a sending job.
Real SMTP not exercised	Emails verified through a local catcher.	Configure a production SMTP provider and send a test.
Single-browser verification	Chrome only.	Cross-browser and accessibility testing (screen readers).
Product copy is generic	Descriptions are the original category text; no certifications or standards are claimed.	Add real descriptions, features and specifications.
Development-only advisories	Five advisories remain in lint tooling; nothing ships to production.	Revisit when the tooling publishes a fix that does not downgrade Next.js.

Table 8. Known limitations.

13. Conclusion

The project turns a static brochure site into a maintainable, secure web application whose content, leads and usage data live in one database. It meets each objective set out in section 3: content is database-driven and live; every product has its own page; enquiries are validated, numbered, notified, trackable and protected from spam; administration is role-based and audited; analytics respect privacy; and the whole system can be run with one command and checked with 300 automated tests.

Compared with the original, the clearest gains are in lead handling (reference numbers, acknowledgements, tracking and a triage workflow), discoverability (individual pages, sitemap, structured data and search) and operability (editing without a developer, backup and restore, audit and monitoring). The principal remaining work is a graphical administration interface so that non-technical staff can use these capabilities without the API.

Appendix A. API summary and how to run

A.1 API operations (72)

Group	Operations
Public content (12)	`GET /api/settings · stats · products · products/{slug} · product-groups · surveys · surveys/{slug} · training · gallery · testimonials · leaders · logos`
Public utility (4)	`GET /api/search · catalogue · media/{id} · health`
Public enquiries and engagement (11)	`GET /api/enquiries/token`; `POST /api/enquiries · enquiries/track · testimonials · newsletter/subscribe · analytics/events`; `GET+POST /api/newsletter/confirm · unsubscribe`
Scheduled (1)	`POST /api/cron/notifications` (secret-protected)
Admin (45)	Authentication and sessions; users; enquiries (list, get, update, delete, notes, resend, bulk, CSV); content for 8 types (list, create, get, update, delete, reorder, bulk); settings; media; subscribers; dashboard; analytics; audit; export, import and cache purge

Table 9. API overview. Full table in features.readme.md.

A.2 Running the project

Requirements: Node 20.9 or later (developed on Node 24). No Docker or system database is needed.

`npm install` then `npm run dev:all`. This starts a local MongoDB, a local mail catcher, seeds the database and runs the site:

- Website: `http://localhost:3000`
- Mail catcher (every email the site sends): `http://127.0.0.1:8025`
- Database: `mongodb://127.0.0.1:27017/apex`
- Tests: `npm test` · type-check: `npm run typecheck` · lint: `npm run lint`

A.3 Project documents

`README.md` (how to run), `features.readme.md` (complete feature inventory), `.env.example` (every setting) and the source in `src/`, `scripts/` and `tests/`.

Report prepared by Manmay Chakraborty (24BAI1316), 20 October 2026. Observations of the existing website were made from its public pages and scripts on that date and may change as the site is updated.